AGENDA 6

UTILIZANDO MICROSOFT MAUI PARA DESENVOLVIMENTO MOBILE

GEEaD - Grupo de Estudos de Educação a Distância

Centro de Educação Tecnológica Paula Souza

GOVERNO DO ESTADO DE SÃO PAULO

EIXO TECNOLÓGICO DE INFORMAÇÃO E COMUNICAÇÃO

CURSO TÉCNICO EM DESENVOLVIMENTO DE SISTEMAS

PROGRAMAÇÃO MOBILE II

Expediente

Autores:

TIAGO ANTONIO DA SILVA

Revisão Técnica:

KELLY CRISTIANE DE OLIVEIRA DAL POZZO

GUILHERME HENRIQUE GIROLLI

São Paulo – SP, 2024

MERGULHANDO NO TEMA

Depois de uma jornada repleta de muito aprendizado e exploração de conceitos do C# e dos recursos da plataforma .NET MAUI, vamos concluir o aplicativo MinhasCompras, para finalizar vamos realizar alguns ajustes como regionalização do App, facilidades como arrastar para atualizar e daremos aquele capricho no aspecto visual do App.

Vamos iniciar com a regionalização, em outras palavras, para que o App adote os aspectos da nossa cultura, em especial a formatação de moeda, adicionando o R$ aos valores monetários.

Vamos iniciar compreendendo os conceitos!

CurrentCulture

O CurrentCulture representa a configuração de localização e formatação usada pelo aplicativo para processar informações de números, datas e moedas. Essa cultura influencia como esses valores são apresentados e interpretados.

Em um aplicativo .NET MAUI, o uso de CultureInfo.CurrentCulture e CultureInfo.CurrentUICulture é essencial para definir a cultura do aplicativo, garantindo que ele exiba textos, formatações de números, moedas e datas conforme as convenções do idioma e da região desejados.

O CurrentCulture define as configurações culturais que um aplicativo usa para formatar e interpretar dados como datas, números e moedas. Cada cultura é representada por um código, como:

- "pt-BR" → Português do Brasil
- "en-US" → Inglês dos EUA
- “fr-FR" → Francês da França

Além do estilo moeda o CurrentCulture também pode ser utilizado para:

Formatos de número

- 1234.56 em "en-US" → 1,234.56
- 1234.56 em "pt-BR" → 1.234,56

- **Formatos de data**
  - DateTime.Now.ToString() em "en-US" → 3/20/2025 3:45 PM
  - DateTime.Now.ToString() em "pt-BR" → 20/03/2025 15:45

Sendo muito importante para garantir consistência e usabilidade, especialmente quando se trabalha com valores financeiros e datas. No aplicativo MinhasCompras, definir CurrentCulture = new CultureInfo("pt-BR")

garante que os números sejam exibidos com vírgula como separador decimal, seguindo o formato 1.234,56. Além disso, as datas são apresentadas no padrão brasileiro dd/MM/yyyy, e os valores monetários aparecem corretamente com o símbolo R$ em vez de $, proporcionando uma experiência mais intuitiva para os usuários no Brasil.

```
public App()
{
    InitializeComponent();

     Thread.CurrentThread.CurrentCulture = new CultureInfo("pt-BR");

    //MainPage = new AppShell();
    MainPage = new NavigationPage(new Views.ListaProduto());
}
```

Vamos visualizar outro exemplo:

```
using System.Globalization;
using System.Threading;

public partial class App : Application
{
    public App()
    {
        InitializeComponent();

        // Definindo a cultura para Português do Brasil
        var cultura = new CultureInfo("pt-BR");

        // Definindo para a aplicação inteira
        Thread.CurrentThread.CurrentCulture = cultura;
        Thread.CurrentThread.CurrentUICulture = cultura;
    }
}
```

• **CurrentCulture:** refere-se formatação de números, moedas, datas e horas.

• **CurrentUICulture:** refere-se a escolha de recursos localizados, como textos e mensagens na interface do usuário.

O uso de CurrentCulture e CurrentUICulture permite personalizar o comportamento do app para diferentes idiomas e regiões, melhorando a experiência do usuário e garantindo que formatações e textos estejam adequados.

Currency Format

O Currency Format, representado pelo formato **:C** é um especificador de formato numérico do .NET, utilizado para exibir um número no formato de moeda. Ele automaticamente adiciona o símbolo monetário correto e aplica a formatação de separadores de milhares e decimais de acordo com a cultura definida no CurrentCulture.

Como funciona?

Adicionando ao código C#.

```
double valor = 1234.56;

Console.WriteLine($"O valor é {valor:C}");  // Usando interpolação de string
Console.WriteLine(valor.ToString("C"));     // Usando ToString()
```

Se a cultura (CurrentCulture) estiver definida como "pt-BR", o resultado será:

O valor é R$ 1.234,56

Se a cultura (CurrentCulture) estiver definida como "en-US", o resultado será:

O valor é $1,234.56

No projeto o formato **:c** foi aplicado no cálculo do total dos produtos listados e exibi-lo no formato correto de moeda:

Arquivo ListaProduto.xaml.cs:

```
private void ToolbarItem_Clicked_1(object sender, EventArgs e)
{
    double soma = lista.Sum(i => i.Total);

    string msg = $"O total é {soma:C}";

    DisplayAlert("Total dos Produtos", msg, "OK");
}
```

O **:C** dentro da interpolação de string ($"...") formata soma como moeda, exibindo o símbolo correto dependendo da cultura definida no projeto.

Mecanismo Pull to Refresh

O mecanismo Pull to Refresh permite ao usuário atualizar o conteúdo de uma ListView puxando a lista para baixo. Ele é implementado através do RefreshView, um contêiner que envolve a ListView e fornece suporte à funcionalidade de atualização.

Para implementar o Pull to Refresh de maneira simples, usamos o evento Refreshing diretamente no código-behind:

1. Envolver a ListView dentro de um RefreshView.
2. Manipular o evento Refreshing no código-behind para atualizar os dados.
3. Definir IsRefreshing como false após concluir a atualização.

O Pull to Refresh foi implementado no projeto para permitir que o usuário atualize manualmente a lista de produtos, garantindo que sempre veja os dados mais recentes.

1. Remove todos os itens da lista atual para recarregá-los:

```
lista.Clear();
```

2. Busca a lista de produtos armazenada no banco SQLite:

```
List<Produto> tmp = await App.Db.GetAll();
```

3. Atualiza a interface com os produtos mais recentes:

```
tmp.ForEach(i => lista.Add(i));
```

4. Finaliza a animação de carregamento:

```
lst_produtos.IsRefreshing = false;
```

Veja o código completo:

```
private async void lst_produtos_Refreshing(object sender, EventArgs e)
{
    try
    {
        lista.Clear();
        List<Produto> tmp = await App.Db.GetAll();
        tmp.ForEach(i => lista.Add(i));
    }
    catch (Exception ex)
    {
        await DisplayAlert("Ops", ex.Message, "OK");
    }
    finally
    {
        lst_produtos.IsRefreshing = false;
    }
}
```

O código acima funciona porque há um componente CollectionView ou ListView na interface (.xaml) que tem a propriedade IsPullToRefreshEnabled="True".

```
<ListView x:Name="lst_produtos"
          IsPullToRefreshEnabled="True"
          RefreshCommand="{Binding RefreshCommand}"
          IsRefreshing="{Binding IsRefreshing, Mode=TwoWay}"
          ItemSelected="lst_produtos_ItemSelected">
</ListView>
```

Definições de SplashScreen e Ícone do Aplicativo

No .NET MAUI, a Splash Screen e o Ícone do Aplicativo são configurados de forma centralizada usando imagens vetoriais (SVG ou PNG) e um único arquivo MauiProgram.cs, permitindo que sejam gerados automaticamente para diferentes plataformas (Android, iOS, Windows e macOS).

O ícone do app também é configurado centralmente no MauiProgram.cs. Ele deve ser um arquivo SVG ou PNG de alta qualidade, sendo que:

1. O .NET MAUI gera automaticamente variações do ícone para Android, iOS, Windows e macOS.
2. Se usar SVG, o .NET MAUI ajusta automaticamente os tamanhos.
3. Para ícones PNG, recomenda-se um tamanho mínimo de 512x512px.

O .NET MAUI simplifica a configuração da Splash Screen e do Ícone ao gerar automaticamente imagens para todas as plataformas a partir de um único arquivo. Isso reduz a complexidade do desenvolvimento e mantém a consistência visual do app.
A SplashScreen é configurada no arquivo MauiProgram.cs, no método CreateMauiApp().
Veja um exemplo:

```
builder.UseMauiApp<App>()
       .UseMauiSplashScreen("Resources/Splash/splashscreen.png");
```

Se a SplashScreen não foi configurada no código, ela pode estar definida no arquivo de recursos do projeto, especificamente em:

- Platforms/Android/mipmap → Para Android.
- Platforms/iOS/Assets.xcassets → Para iOS.
- Resources/Splash → Diretório padrão de imagens.

Já o ícone do aplicativo é definido no arquivo .csproj, que foi carregado como MauiAppMinhasCompras.csproj. A configuração padrão do ícone e da SplashScreen costuma estar nesse arquivo da seguinte forma:

```
<MauiIcon Include="Resources\appicon.svg" />
<MauiSplashScreen Include="Resources\splashscreen.svg" />
```

Assista o vídeo apresentando o passo a passo para criação do aplicativo:

Disponível em: https://www.youtube.com/watch?v=T1BvGYBo-Jc

CÓDIGO-FONTE DA AGENDA:

https://github.com/tiagotas/MauiAppMinhasCompras/commit/8f49c267a4ea3f433273e146337cea80b798abe1